

A&I

android and ios develop.

Airflow

2026 A&I 4th

구승민

목차

1. Airflow 이해하기

- 노트북에서 파이프라인으로 넘어가는 이유

2. 실습 환경 구성

- Docker와 WSL2로 실행 환경 준비하기

3. SASRec 파이프라인 실습

- 학습을 자동 실행하고 WandB로 관측하기

1. Airflow 이해하기

- 노트북 밖으로 나가봅니다

노트북 실행의 한계

5주차까지의 방식

- 노트북에서 셀을 위에서 아래로 직접 실행
- 한 번 학습하고 결과를 확인하면 끝

반복하려는 순간 생기는 문제

- 사람이 순서대로 다시 실행해야 함
- 중간에 실패하면 처음부터 다시
- 어떤 설정으로 학습했는지 기록이 사라짐

이번 주의 질문

- SASRec을 매주 새 데이터로 다시 학습시켜야 한다면?

워크플로 오케스트레이션

현업의 요구

- 작업들이 정해진 의존 관계에 따라 자동 실행
- 실패하면 자동으로 재시도
- 모든 실행 이력이 남아 추적 가능

이 요구를 해결하는 도구 범주

- 워크플로 오케스트레이션 (Workflow Orchestration)
- 사실상의 표준이 Apache Airflow

Airflow 소개

Apache Airflow

- 2014년 Airbnb에서 개발, Apache 최상위 프로젝트
- 데이터 엔지니어링 분야에서 가장 널리 채택

핵심 철학 — Workflow as Code

- 파이프라인 자체가 파이썬 코드
- 버전 관리와 재사용이 가능

강의 기준 버전

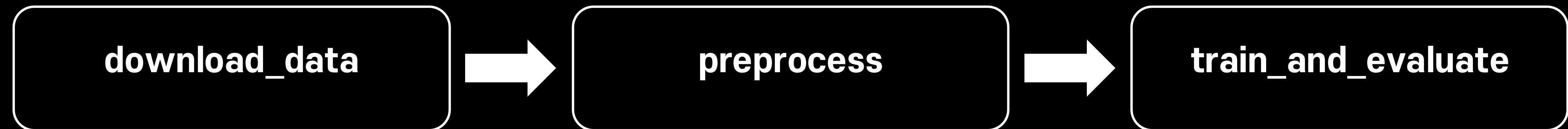
- Airflow 3.x — 새 UI, DAG 버전 관리, 이벤트 기반 스케줄링

DAG의 구조

DAG (Directed Acyclic Graph)

- 작업 간 의존 관계를 화살표로 표현, 순환은 없음
- "순서가 정해진 작업들의 설계도"

오늘 만들 DAG



Task와 TaskFlow API

Task — DAG를 구성하는 개별 작업 단위

TaskFlow API — 최신 표준 작성법

- 함수에 @task, 전체 흐름 함수에 @dag 데코레이터
- 함수의 반환값이 다음 Task의 입력으로 자동 전달 (XCom)

코드 한 줄이 곧 의존 관계

`report(preprocess(download_data()))`

- 호출처럼 보이지만 실행이 아니라 "설계도 그리기"

Airflow 아키텍처

역할 분담 — Docker Compose가 한 번에 띄워줌

- Scheduler — DAG 폴더 감시, 실행 시점에 Task를 대기열로
- API Server — 웹 UI, 모니터링과 수동 트리거
- Worker — Task를 실제로 실행
- Metadata DB — 모든 실행 이력과 상태 저장

Scheduler

API Server

Worker

Metadata DB

스케줄과 트리거

schedule — 언제 실행할 것인가

`@dag(schedule=None, start_date=datetime(2026, 1, 1), catchup=False)`

- 크론식을 넣으면 주기 실행 — 예: "0 3 * * *" 매일 새벽 3시
- 이번 실행은 None — UI에서 수동 트리거

catchup — 과거 구간을 소급 실행할지

- 실행에서는 False로 꺼 둬

retries — 실패한 Task 자동 재시도

- 노트북에는 없는 능력

WandB 소개

Weights & Biases — 실험 추적 도구

- 학습 코드에 단 두 줄이면 동작

```
wandb.init(project=..., config=하이퍼파라미터)  
wandb.log({"loss": ..., "HR@10": ...})
```

print 로그와의 차이

- 기록이 휘발되지 않음
- 하이퍼파라미터와 결과가 한 묶음으로 저장
- 여러 실험(run)을 겹쳐 비교 — 개인 무료 플랜으로 충분

Airflow와 WandB의 역할

Airflow — 실행

- 언제, 어떤 순서로 실행할 것인가

WandB — 관측

- 학습이 어떻게 진행되었는가

이번 강의의 전부

- 자동 실행되는 파이프라인 + 기록되는 실험
- 이 둘이 곧 MLOps의 최소 단위

2. 실습 환경 구성

- 한 번만 제대로 구성하면 됩니다

환경 구성 전체 흐름

세 단계

- 1) Docker Desktop 준비 — Windows는 WSL2부터
- 2) 6주차 폴더에서 .env 작성과 compose 수정
- 3) 이미지 빌드 후 docker compose up

OS별 차이

- macOS — Docker Desktop 설치만 하면 됨
- Windows — WSL2 설치 후 Docker Desktop 연동 (다음 장)
- 이후 명령은 모든 OS에서 동일

사전 준비물

설치

- Docker Desktop — docker.com에서 다운로드
- Windows는 WSL2 설치가 선행 (다음 장)

가입

- wandb.ai 무료 계정 생성
- 프로필에서 API Key 발급 — 잠시 후 .env에 넣음

설치 확인

```
docker --version  
docker compose version
```

WSL2 설치 — Windows

PowerShell을 관리자 권한으로 실행

`wsl --install`

- 재부팅 후 Ubuntu에서 리눅스 사용자명과 암호 생성

설치 확인

`wsl -l -v`

- VERSION이 2인지 확인

실패할 때 점검 세 가지

- Windows 10 2004 이상 / BIOS 가상화(VT-x, AMD-V) 활성화 / "Virtual Machine Platform" 기능 켜기

Docker Desktop 연동 — Windows

설치할 때

- "Use WSL 2 based engine" 체크

설치한 뒤

- Settings → Resources → WSL Integration → Ubuntu 토글 ON

확인은 Ubuntu 터미널에서

`docker --version`

- 이후 모든 명령은 Ubuntu(WSL) 터미널에서 실행

리포지토리 클론 — Windows

WSL 홈 디렉터리에 클론

```
cd ~
```

```
git clone https://github.com/Team-Anl/A-AND-I-4TH-AI-CODE-LAB.git
```

```
cd A-AND-I-4TH-AI-CODE-LAB/6주차
```

왜 /mnt/c가 아니라 홈인가

- 리눅스 파일시스템이라 빠르고 권한 문제가 없음
- macOS는 평소처럼 원하는 위치에 클론하면 됨

.env 작성 — 공통

공식 compose 파일 받기 (6주차 폴더에서)

```
curl -LfO https://airflow.apache.org/docs/apache-airflow/stable/docker-compose.yaml
```

.env 파일 — 키는 코드가 아니라 환경변수로

```
AIRFLOW_UID=1000
```

```
AIRFLOW_IMAGE_NAME=airflow-sasrec-lab:6
```

```
WANDB_API_KEY=발급받은-키
```

- AIRFLOW_UID는 id -u 값 (WSL 기본 1000)
- API 키를 코드에 적지 않는 것이 협업의 기본기

docker-compose.yaml 수정 — 공통

x-airflow-common 블록에 세 가지

- environment에 한 줄 추가 — WandB 키 주입
WANDB_API_KEY: \${WANDB_API_KEY:-}
- volumes에 한 줄 추가 — 산출물을 호스트에서 보기
- \${AIRFLOW_PROJ_DIR:-.}/data:/opt/airflow/data
- 예제 DAG 끄기 — 실습 DAG만 보이게
AIRFLOW__CORE__LOAD_EXAMPLES: 'false'

이미지 빌드와 기동 — 공통

실습용 이미지 빌드 — 동봉된 Dockerfile 사용

`docker build -t airflow-sasrec-lab:6 - < Dockerfile`

- `stdin(-)` 방식인 이유 — 폴더 경로의 한글로 인한 빌드 이슈 회피

초기화 후 기동

`docker compose up airflow-init`

`docker compose up`

끄기 / 다시 켜기

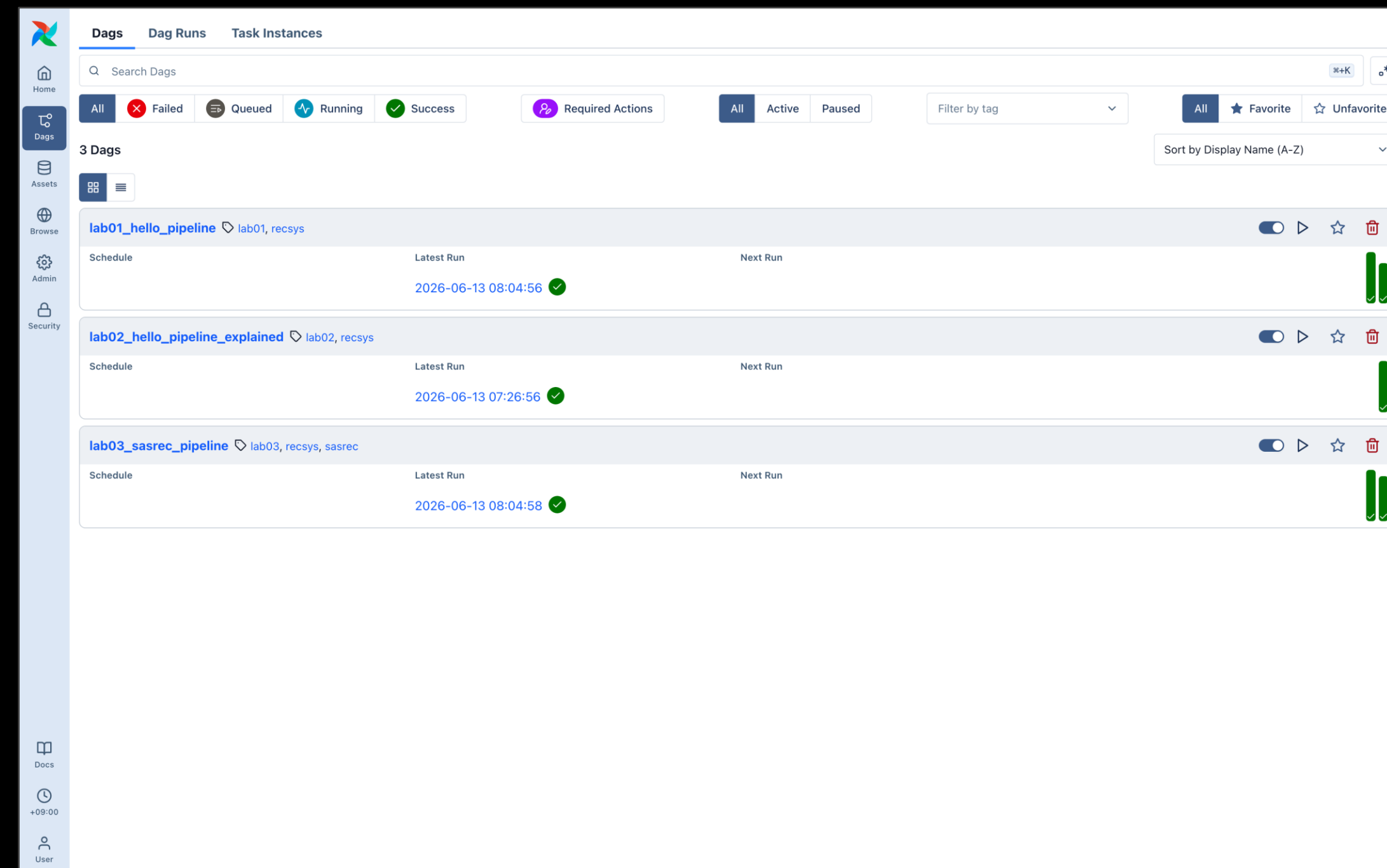
`docker compose down`

`docker compose up -d`

Airflow UI 접속

http://localhost:8080

- 기본 계정 airflow / airflow
- DAG 목록에 lab01, lab02, lab03 세 개가 보이면 성공



3. SASRec 파이프라인 실습

- 직접 굴러보며 확인합니다

실습 구성

세 단계로 진행

- 실습 1 — lab_01: 첫 DAG를 트리거해 동작 관찰 (따라하기)
- 실습 2 — lab_02: 같은 DAG의 코드를 한 줄씩 이해 (코드 해설)
- 실습 3 — lab_03: SASRec 학습 파이프라인 실행 (확장)

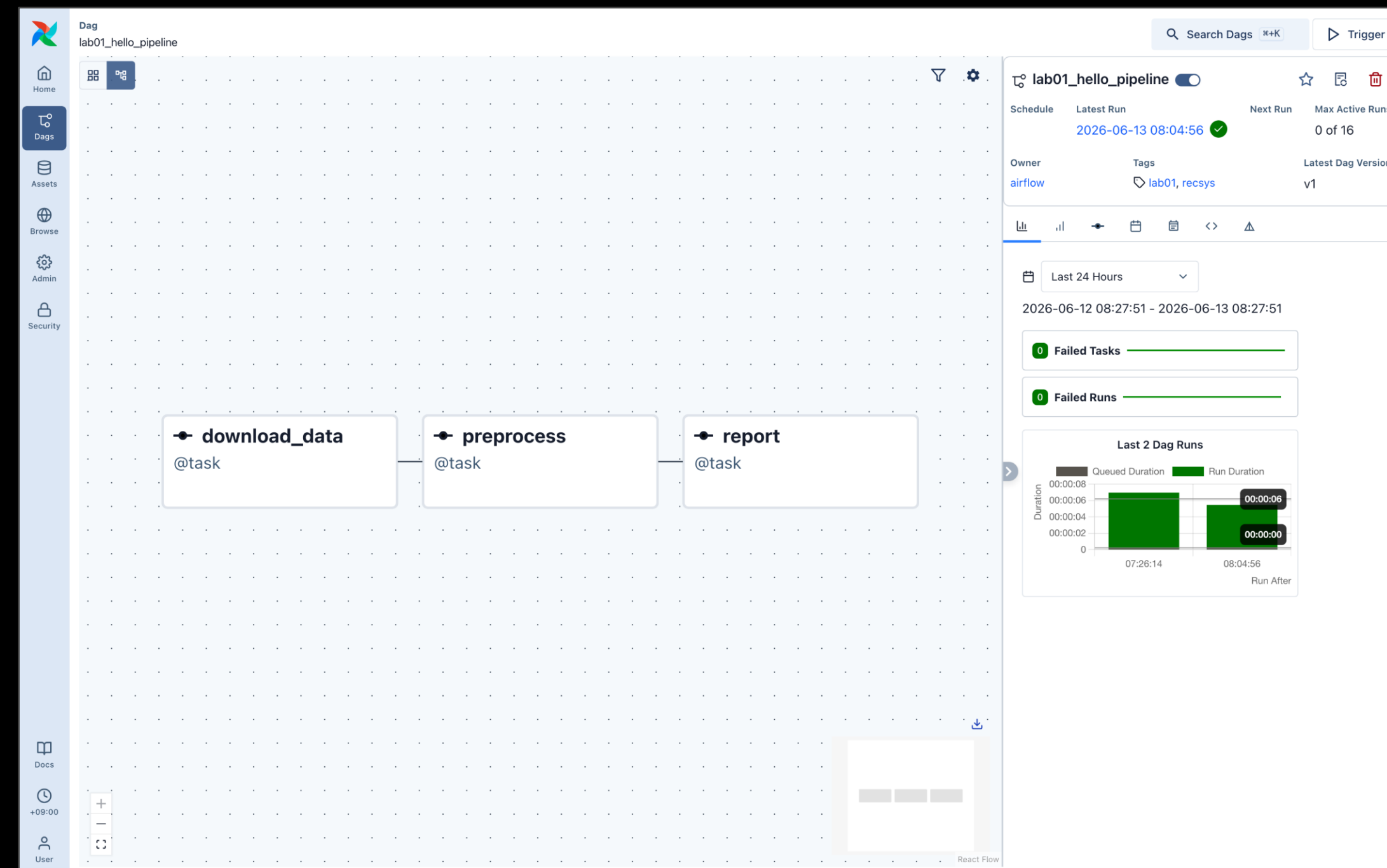
자료 위치

- github.com/Team-Anl/A-AND-I-4TH-AI-CODE-LAB → 6주차/dags
- 자세한 진행 안내는 6주차 README 참고

실습 1 — 첫 DAG 실행

lab01_hello_pipeline

- 토글을 켜고 Trigger → 그래프 뷰에서 Task가 순서대로 초록색으로
- Task 클릭 → Logs 탭에서 print 출력 확인 — 기록이 남는다



실습 2 — 코드 해설

lab_02_explain.py — 같은 코드, 해설 주석 포함 짚어볼 포인트

- @dag 파라미터 — schedule=None과 catchup=False의 의미
- 반환값을 인자로 쓰면 그대로 의존 관계가 됨 (XCom)
report(preprocess(download_data()))
- 무거운 import는 함수 안에 — 파일 파싱을 가볍게 유지
- print는 사라지지 않고 Task 로그로 저장

실습 3 — SASRec 파이프라인

lab03_sasrec_pipeline — 5주차 코드를 그대로 재사용

- **download_data** — MovieLens 내려받기 (100K 기본, 1M은 주식 한 줄 교체)
- **preprocess** — timestamp 정렬, 시퀀스 구성, leave-one-out 분할
- **train_and_evaluate** — SASRec 학습과 WandB 기록



실습 3 — 핵심 코드

학습 Task 안에서 일어나는 일

```
run = wandb.init(project="club-airflow-sasrec", config=CONFIG)
run.log({"loss": ..., "HR@10": ..., "NDCG@10": ...})
```

- CONFIG의 hidden, epochs를 바꾸면 새로운 실험(run)이 됨

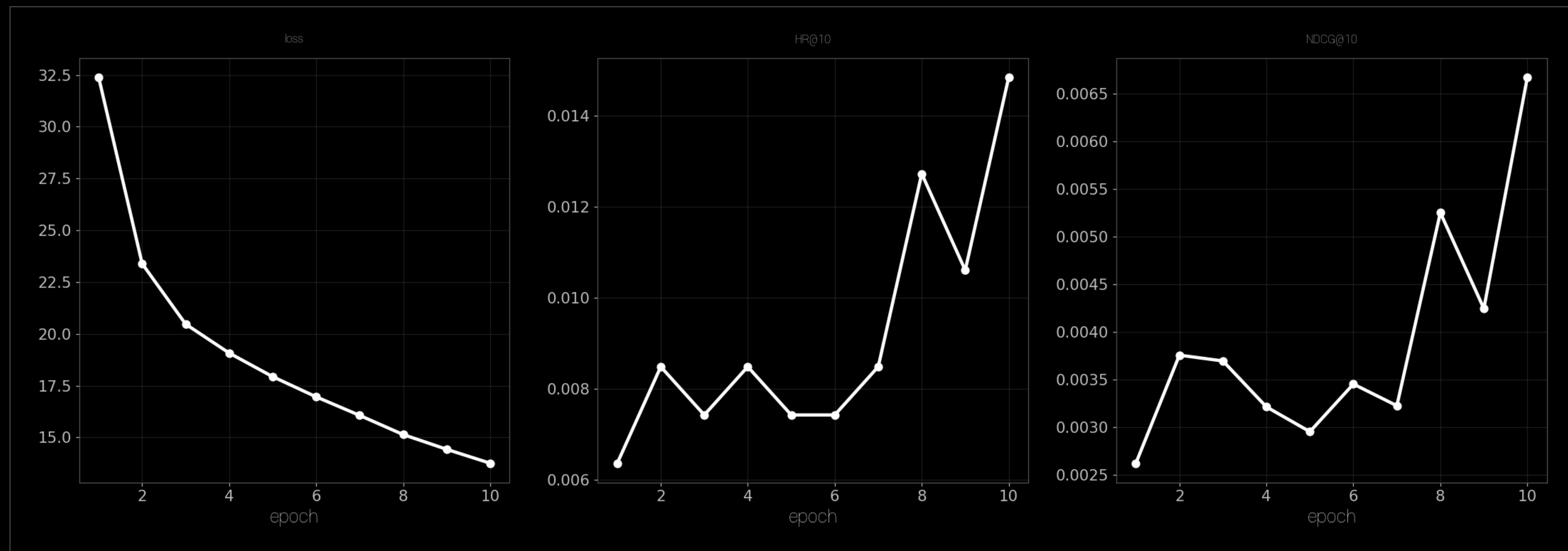
평가 — leave-one-out

- 각 사용자의 마지막 아이템은 test, 그 앞은 valid로 남겨 두고 학습
- HR@10 — 정답이 상위 10개 안에 든 비율
- NDCG@10 — 정답의 순위까지 반영한 점수

WandB 대시보드

wandb.ai → club-airflow-sasrec 프로젝트

- epoch마다 loss, HR@10, NDCG@10 곡선이 실시간으로 쌓임
- 어떤 설정(config)으로 학습했는지 run마다 함께 저장



심화 실습

실험 비교 — WandB를 쓰는 진짜 이유

- CONFIG의 hidden을 64에서 128로 바꿔 다시 트리거
- 대시보드에서 두 run을 겹쳐 곡선 비교

실패와 재시도

- Task를 일부러 실패시켜 retries 동작 관찰
- 노트북과 달리 파이프라인은 실패를 스스로 복구한다

마무리

오늘 만든 것 — MLOps의 최소 단위

- 자동 실행되는 학습 파이프라인 + 기록되는 실험

확장 방향

- schedule로 주기 학습 — 매주 자동 재학습
- 새 데이터 도착 시 실행되는 이벤트 기반 트리거 (Airflow 3)
- 학습된 모델을 API로 서빙 — 추후 주제

다음 주

- 논문 탐색과 발표 — 오늘 골려본 SASRec 원논문 읽기부터

Q & A

End of Document